

Project 3: Active Learning for Learning-to-Defer

Human-Centric Artificial Intelligence

AG News topic classification with a human-AI team

Generated from the experiment artifacts on 2026-09-10

Experimental setup

All experiments use the AG News dataset (fancyzhx/ag_news): 120,000 training and 7,600 test news articles, evenly distributed over four topics (World, Sports, Business, Sci/Tech). The official training set is split once, stratified with a fixed seed, into 110,000 training and 10,000 validation articles. Every design decision that needs tuning uses only the validation split: the deferral threshold in Task 3, and all monitoring during active learning in Task 4. The official test set is reserved for final evaluation, so each reported test number is computed once.

The code is organised as an offline pipeline (`python -m project3.experiments.run_all`) that writes metrics, figures and this report to disk; the project web page displays those artifacts, offers this report for download, and adds an interactive mode where a person can play the expert (Task 5).

Task 1: Baseline classifier

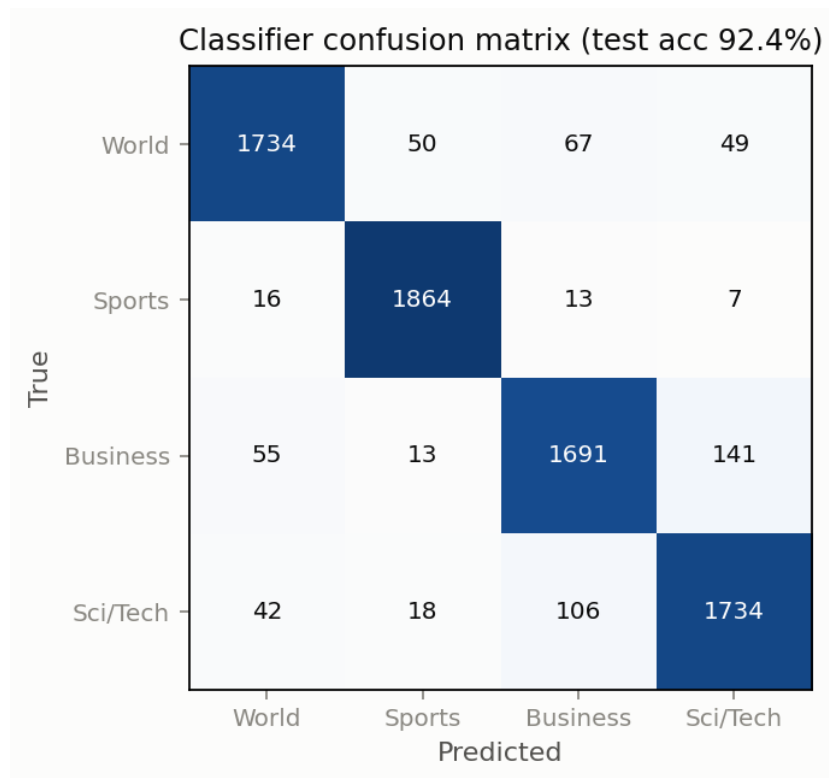
Design

The baseline is a linear classifier over sparse TF-IDF features: word 1-2 grams, sublinear term frequency, 300k features, followed by multinomial logistic regression ($C=4$). A fine-tuned transformer was the obvious alternative and was not used. The linear model trains on 110k documents in about half a minute on CPU, which keeps the rejector training, the nine active-learning sweeps and the interactive mode cheap enough to re-run. It also supplies the per-example confidence scores the deferral system depends on, and logistic regression's probability estimates are already reasonably calibrated. A deep model might gain a little accuracy, but that gain would not change any of the human-AI comparisons studied here.

Results

The classifier reaches 92.41% accuracy on the test set (validation: 92.77%). The errors are not spread evenly. Sports is almost always right, while Business and Sci/Tech are confused with each other more often than any other pair. That is the region where a complementary expert could help.

Class	Precision	Recall	F1	Support
World	0.939	0.913	0.926	1900
Sports	0.958	0.981	0.970	1900
Business	0.901	0.890	0.895	1900
Sci/Tech	0.898	0.913	0.905	1900



Task 2: Simulated experts

Design

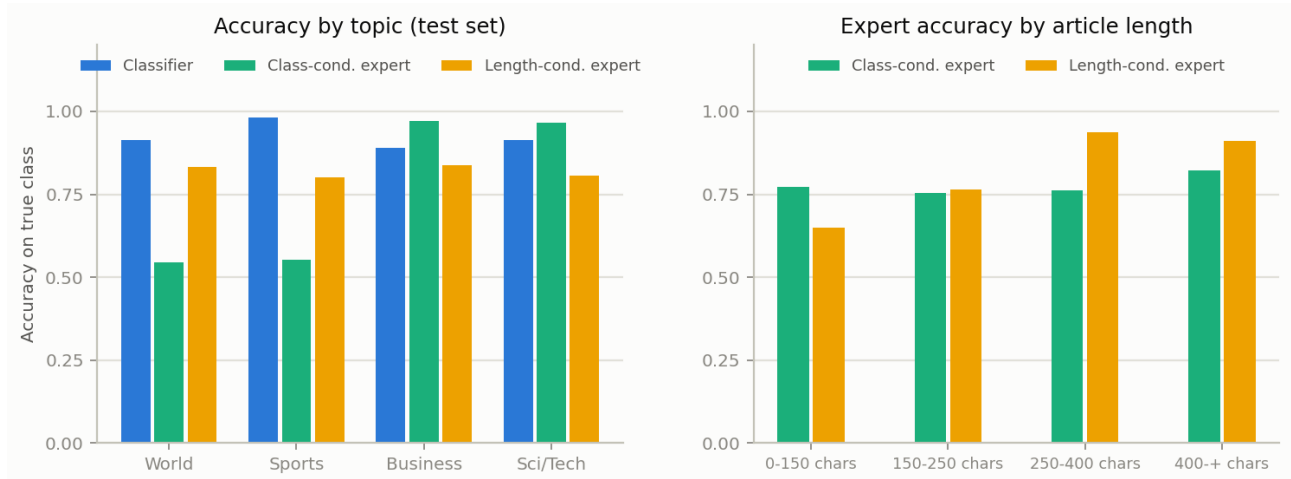
Both experts are imperfect by construction and competent only in part of the input space. They are deterministic: each answer is derived from a hash of the article's row key, so querying the same article twice always returns the same answer. That consistency is needed because Tasks 3, 4 and 5 all query the same expert. When an expert errs, it prefers a plausible confusion (e.g. Business vs Sci/Tech) rather than a uniformly random topic.

The primary expert is class-conditional: Class-conditional expert: 97% correct on [Business, Sci/Tech], 55% on the other topics. It stands in for a business and technology specialist, and its strong classes (Business, Sci/Tech) are the two the Task 1 classifier handles worst, so the two team members complement each other. The second expert is length-conditional: Length-conditional expert: 93% correct on articles ≥ 215 chars, 62% on shorter ones. It stands in for a reader who judges long articles reliably but guesses on short snippets, and its competence region does not follow class boundaries at all.

Results and analysis

On the test set the class-conditional expert reaches 75.9% overall, well below the classifier's 92.4%. On its strong classes it does better: 97.2% on Business and 96.6% on Sci/Tech against the classifier's 89.0% and 91.3%. Its weakness is everything else (54.5% on World, 55.3% on Sports). The length-conditional expert reaches 81.9% overall, with accuracy rising as articles get longer. Neither expert could replace the classifier on its own, but a deferral policy that knows these profiles can route each article to whichever team member handles it better.

Expert	Overall	World	Sports	Business	Sci/Tech
Class-conditional	75.9%	54.5%	55.3%	97.2%	96.6%
Length-conditional	81.9%	83.3%	80.1%	83.6%	80.7%



Task 3: Learning to defer

Method

With expert labels available for the whole training set, the system learns when deferral is beneficial with a staged (post-hoc rejector) approach. A binary rejector $g(x)$ is trained on the same TF-IDF features to predict the probability that the expert answers x correctly, using the observed expert answers on the training set. The team then defers exactly when the expert looks more reliable than the classifier: defer iff $g(x) - \max_y p(y|x) > \tau$. The threshold τ is chosen on the validation split (selected value: $\tau = 0.04$).

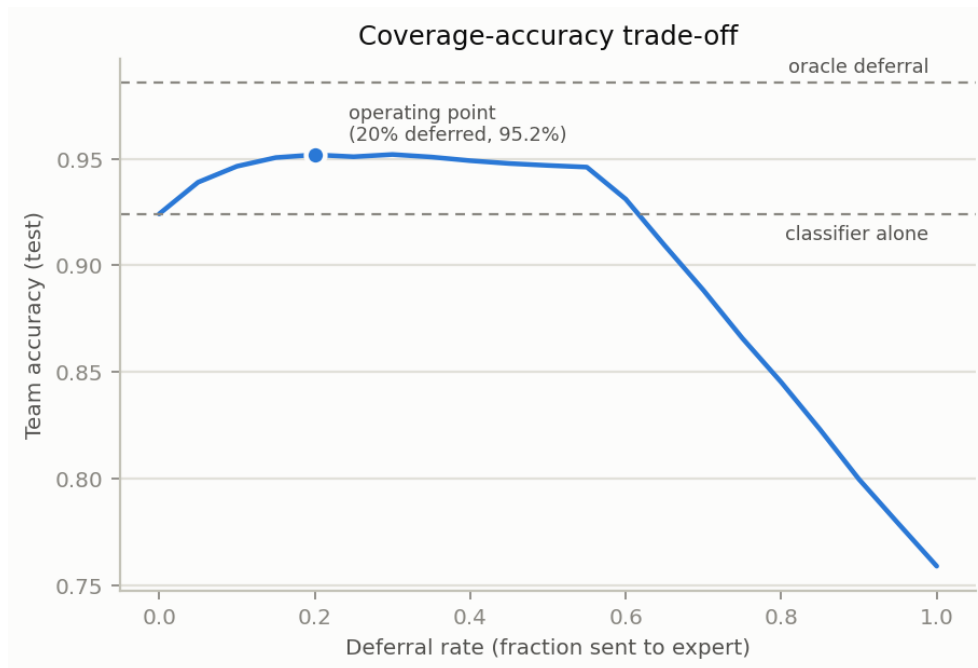
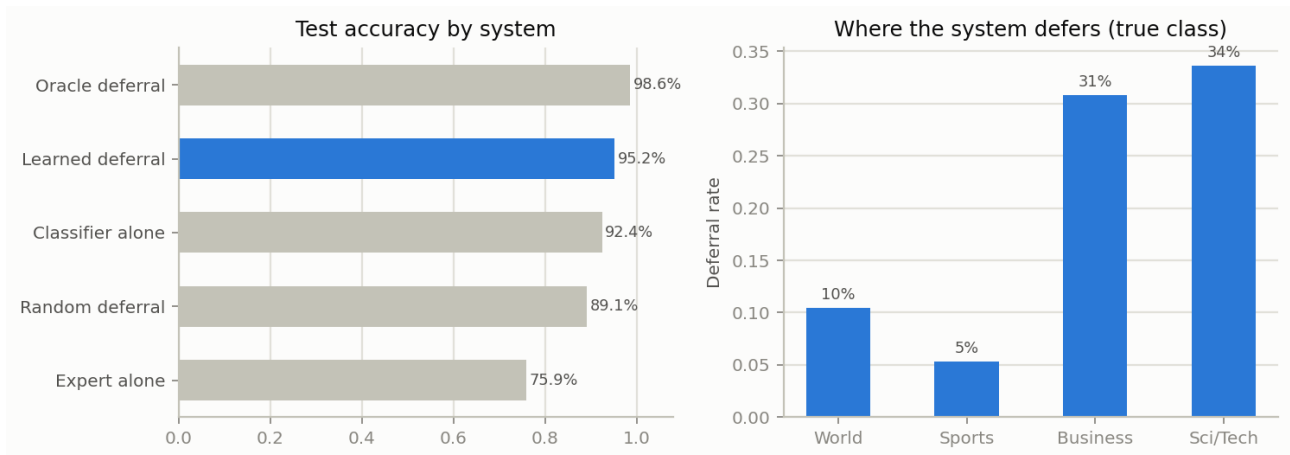
The main alternative is a joint surrogate loss in the style of Mozannar & Sontag (2020), a $(K+1)$ -way classifier with a defer option trained under a specially weighted loss. It cannot be expressed with scikit-learn's estimators, which allow no custom per-class loss weighting. The staged approach also suits the analysis better, since $g(x)$ is an explicit model of the expert's competence that can be inspected, and Task 4 goes on to learn it actively.

Results

The team reaches 95.17% test accuracy at 79.9% coverage: 1,524 of the 7,600 test articles (20.1%) are sent to the expert and the classifier answers the rest. That is above the classifier alone at 92.41%, the expert alone at 75.88%, and random deferral at the same rate, 89.14%. The oracle that defers whenever it helps reaches 98.57%.

The split of work is a good one in both directions. The classifier gets 96.99% on the articles it keeps, and the expert gets 87.93% on the ones sent over, each well above what the other team member manages on that same share. The deferral pattern also tracks the expert's competence: 31% of Business articles and 34% of Sci/Tech go to the expert against only 5% of Sports, which is the expert's strong region, and nothing in the training signal named it. Against the oracle should-defer set (classifier wrong and expert right), deferral recall is 68.2% and precision 20.9%. Low precision is expected at this operating point, since deferring an article the classifier would also have got right costs the team nothing where the expert is strong.

System	Test accuracy
Expert alone	75.88%
Random deferral (same rate)	89.14%
Classifier alone	92.41%
Learned deferral (team)	95.17%
Oracle deferral (upper bound)	98.57%



The coverage-accuracy curve defers the top-q fraction by rejector score. Team accuracy peaks around 20-30% deferral and falls off slowly from there toward the expert-alone end, so an operator can pick any workload split for the human expert without retraining.

Task 4: Active learning for expert competence discovery

Setting and strategies

Now no expert labels exist up front. The classifier stays fixed (it already has all gold labels); what is missing is the expert's competence profile, so only the rejector must be learned, from answers obtained by querying the expert. Queries are selected in batches of 100 from a pool of 30,000 training articles; after each batch the rejector is refit on all answers so far and the team is evaluated on the validation split with the fixed rule defer iff $g(x) > \max_y p(y|x)$. Budgets from 25 to 5,000 queries are checkpointed, with 3 seeds per strategy.

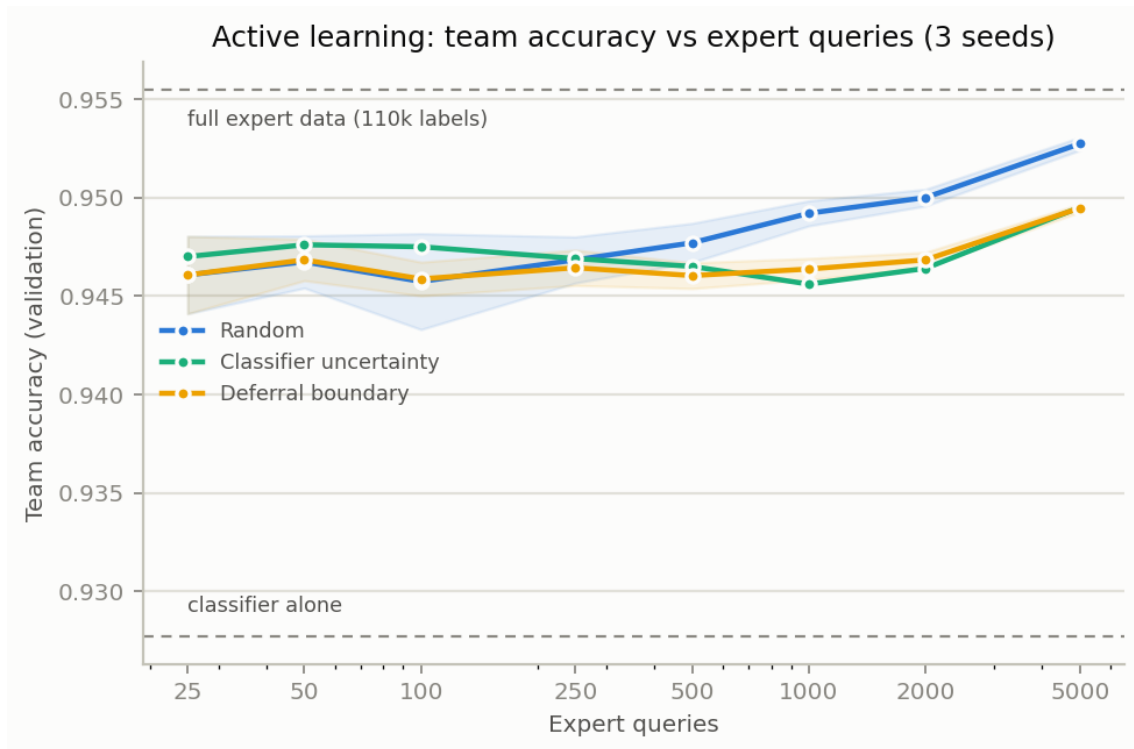
Three query strategies are compared. Random sampling is the baseline. Classifier-uncertainty sampling queries where the classifier is least confident. It is the classic active-learning heuristic, and it is included because deferral candidates tend to sit in that region. The proposed strategy, deferral-boundary sampling, queries where $|g(x) - \max_y p(y|x)|$ is smallest, so the expert's answer is most likely to flip a defer/keep decision. It uses a first random batch and keeps 10% of each later batch random.

Results

Competence discovery turns out to be cheap. With 25 expert queries every strategy already lifts the team to about 94.6% on validation, against 92.77% for the classifier alone and 95.55% for a rejector trained on all 110,000 expert labels. Around 100 queries recover about 65% of the value of full expert supervision, at 0.1% of the labelling cost.

Between strategies, random sampling comes out strongest at large budgets (95.27% at 5,000 queries against 94.95% for deferral-boundary), and at small budgets nothing separates. This expert's competence is class-determined, and class is linearly recoverable from TF-IDF features, so a handful of random examples per topic already pins the profile down. Both targeted strategies also collect a biased sample, of hard or boundary articles, which slightly miscalibrates the rejector's probabilities; since the deferral rule compares $g(x)$ against classifier confidence directly, calibration counts for more here than ranking does. Targeted querying should pay off where competence varies within classes in subtler ways. For this problem, stratified random querying with a few hundred labels is enough.

Strategy	25	50	100	250	500	1000	2000	5000
Random	0.9461	0.9467	0.9457	0.9468	0.9477	0.9492	0.9500	0.9527
Clf. uncertainty	0.9470	0.9476	0.9475	0.9469	0.9465	0.9456	0.9464	0.9495
Deferral boundary	0.9461	0.9468	0.9459	0.9464	0.9460	0.9464	0.9468	0.9495



Task 5: Interactive mode (optional)

The project interface includes a 'Be the expert' mode implementing the optional Task 5. The Task 4 loop runs with a person instead of the simulated expert: the system picks the next article (random warm-up, then deferral-boundary sampling), the user assigns one of the four topics, and the rejector is refit after every answer. That rejector is now a model of the user's own competence. A live panel shows the user's measured accuracy per topic and the estimated team accuracy if the current rejector were deployed with them as the expert. Because the queried articles come from the labeled training set, the user's answers can be scored against gold labels immediately.

Limitations

The experts are simulations, and their competence has a simple structure. As Task 4 showed, that makes competence discovery easy and favours random querying. A real expert's competence drifts over time, varies with effort, and is not a fixed function of the article. The rejector reuses the classifier's TF-IDF representation, so any competence pattern that is invisible in bag-of-words space cannot be learned. Deferral is also evaluated at zero query cost; charging per query would move the operating point along the coverage-accuracy curve. Finally, the staged rejector is not jointly optimal, since the classifier is never retrained to specialise on the region it keeps. That is the cost of an interpretable, sklearn-only design.